

Finaliser un prototype

Lors du développement, il est normal de devoir revenir sur des choix faits en amont. On appelle cela la refactorisation. Cela peut impliquer de modifier la structure du code, d'améliorer la lisibilité ou de simplifier la logique sans changer le comportement externe de l'application.

Voici quelques trucs en vrac avant de mettre en ligne votre prototype.

Normalisez et renommez vos variables correctement

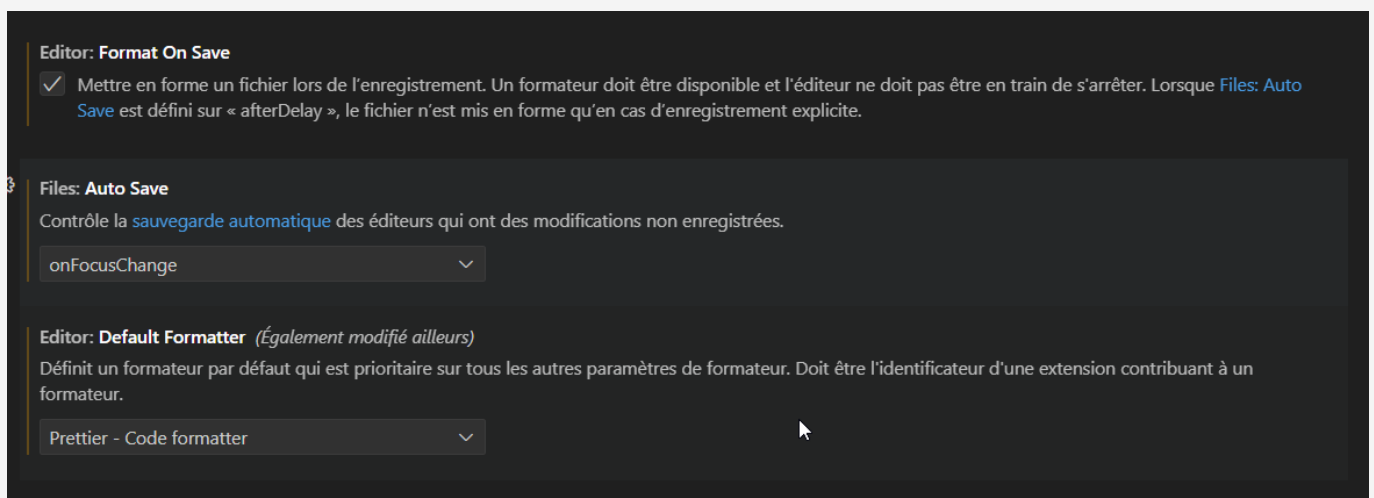
Si vous avez à renommer une variable, une fonction ou une classe, placez le curseur dessus et faites **F2**. Vous pourrez ainsi renommer partout dans le fichier. Si d'autres fichiers utilisent cette variable, fonction ou classe, VSCODE vous proposera "GÉNÉRALEMENT" de les modifier aussi si les fichiers sont ouverts. Sinon, faites une recherche globale avec **CTRL + SHIFT + F** (ou **CMD + SHIFT + F** sur Mac).

Nettoyez votre code

Utilisez Prettier pour formater votre code automatiquement. Cela vous aidera à maintenir une cohérence dans le style de votre code et à éviter les erreurs de syntaxe. De plus, lorsque vous travaillerez en équipe, cela garantira que tout le monde utilise la même indentation et les mêmes conventions de style donc moins de conflits sur Git.

Dans les préférences, activez l'option "Format On Save" pour que Prettier formate votre code à chaque fois que vous enregistrez un fichier. Choisissez votre moteur de formatage préféré (par exemple, "Prettier") dans la liste déroulante. Mettez autosave sur "onFocusChange" pour que vos fichiers se sauvent automatiquement quand vous changez de fenêtre, par exemple quand vous visualisez votre prototype dans un navigateur. Le code se nettoiera automatiquement.

Mettez autosave sur "afterDelay" pour que vos fichiers se sauvent automatiquement lorsque vous tapez mais vous aurez à formater manuellement avec **CTRL + S** (ou **CMD + S** sur Mac).



Utilisez JSDocs pour commenter vos classes et méthodes

JSDocs est une norme de documentation pour JavaScript qui permet de commenter votre code de manière structurée. Cela facilite la compréhension de votre code par d'autres développeurs (ou vous-même dans le futur).

Le raccourci est `/**` + **ENTER** au-dessus d'une fonction, méthode ou classe. Terminez par `*/`.

Indiquez une description et les types de paramètres et de retour pour chaque fonction ou méthode.

En survol, vous pourrez ainsi voir les informations liées à la fonction ou méthode.

```
10
11  /**
12   * Sert à injecter le contenu HTML du widget dans le conteneur spécifié
13   * @param {HTMLElement} conteneur
14   */
15  render(conteneur) {
16    const widgetElement = document.createElement("div");
17    widgetElement.className = "widget";
18    widgetElement.textContent = "This is a widget";
19    conteneur.appendChild(widgetElement);
20  }
21 }
22
23 export default
24 ⚡ @param conteneur
25 new Widget().render(document.body);
26
```

(method) Widget.render(conteneur: HTMLElement): void
Sert à injecter le contenu HTML du widget dans le conteneur spécifié
@param conteneur

```
/**
 * Calcule la somme de deux nombres.
 * @param {number} a - Le premier nombre.
 * @param {number} b - Le deuxième nombre.
 * @returns {number} La somme des deux nombres.
 */
function add(a, b) {
  return a + b;
}
```

Programmation défensive

La programmation défensive consiste à anticiper les erreurs et les problèmes qui pourraient survenir dans un programme. En JavaScript, il est possible de vérifier les types de données, les valeurs de retour des fonctions, les erreurs de syntaxe, etc.

Ex: si vous faites une sélection avec `querySelector` et qu'il y a une chance que l'élément n'existe pas, vous pouvez vérifier si l'élément existe avant de continuer.

```
let elementHTML = document.querySelector("section");

if (elementHTML !== null) {
  elementHTML.classList.add("invisible");
}
```

Un autre façon de faire de la programmation défensive est de gérer les exceptions au début d'une fonction complexe et de quitter la fonction si une erreur survient en utilisant `return` ou `throw`.

```
function calculerSomme(a, b) {  
  if (typeof a !== "number" || typeof b !== "number") {  
    return "Les paramètres doivent être des nombres.";  
  }  
  
  if (a === null || b === null) {  
    return "Les paramètres ne doivent pas être nuls.";  
  }  
  
  return a + b;  
}
```

Lancer une exception ou une erreur

Il est possible de lancer une exception ou une erreur en utilisant le mot-clé `throw`. Cela permet de signaler une erreur et d'arrêter l'exécution du programme. Vous pouvez lancer une exception avec un message d'erreur pour informer les autres développeurs du problème. Généralement, `throw` est utilisé conjointement avec les instructions `try...catch` pour gérer les erreurs.

On peut lancer une exception avec un message d'erreur personnalisé en utilisant le mot-clé `throw` suivi d'un objet `Error` (avec `new`) avec un message d'erreur.

```
function calculerSomme(a, b) {  
  if (typeof a !== "number" || typeof b !== "number") {  
    throw new Error("Les paramètres doivent être des nombres.");  
  }  
  
  if (a === null || b === null) {  
    throw new Error("Les paramètres ne doivent pas être nuls.");  
  }  
  
  return a + b;  
}  
  
try {  
  calculerSomme(5, null);  
} catch (error) {  
  console.error(error.message);  
}
```

Try...catch

Le bloc `try...catch` permet de gérer les erreurs dans un programme. Vous pouvez mettre le code qui risque de générer une erreur dans le bloc `try` et gérer l'erreur dans le bloc `catch`.

Le bloc try contient le code qui risque de générer une erreur. Si une erreur se produit, le programme passe au bloc catch. L'erreur ne sera jamais affichée et le programme continuera à s'exécuter normalement.

Le bloc catch contient le code qui gère l'erreur. Il prend en paramètre l'objet Error qui contient des informations sur l'erreur. Vous pouvez afficher le message d'erreur avec error.message.

```
try {  
  // Code qui risque de générer une erreur  
} catch (error) {  
  console.error(error.message);  
  // Gestion de l'erreur  
}
```

Gestion des erreurs

Il est important de gérer les erreurs dans un programme pour éviter les plantages et les comportements inattendus. Minimale, vous pouvez afficher un message d'erreur dans la console pour informer les autres développeurs du problème.

Conversion de types lorsque nécessaire

En JavaScript, il est fréquent de récupérer une valeur sous forme de chaîne de caractères. Cependant, il est parfois nécessaire de convertir cette valeur en nombre pour effectuer des opérations mathématiques.

Pour convertir une chaîne de caractères en nombre, vous pouvez utiliser les fonctions parseInt ou parseFloat. La fonction parseInt convertit une chaîne de caractères en entier, tandis que la fonction parseFloat convertit une chaîne de caractères en nombre à virgule flottante. La méthode toString permet de convertir un nombre en chaîne de caractères.

```
let nombre = "42";  
let entier = parseInt(nombre);  
let flottant = parseFloat(nombre);  
let chaine = entier.toString();  
  
console.log(entier); // 42  
console.log(flottant); // 42  
console.log(chaine); // "42"
```

Dépannage

En cas de problème, voici quelques étapes de dépannage :

1. **Vérifiez TOUJOURS la console** : Ouvrez la console de votre navigateur (F12) et vérifiez les erreurs dans l'onglet "Console".
2. **Utilisez des console.log** : Ajoutez des instructions console.log dans votre code pour afficher des messages de débogage.
3. **Recherchez des erreurs de syntaxe** : Assurez-vous qu'il n'y a pas d'erreurs de syntaxe dans votre code.

4. **Testez les fonctionnalités individuellement** : Si une fonctionnalité ne fonctionne pas, essayez de la tester isolément.
5. **Consultez la documentation** : Si vous utilisez une bibliothèque ou un framework, consultez la documentation pour des conseils de dépannage. Il n'y a pas seulement ChatGPT dans la vie pour expliquer.

Mettre en ligne un prototype JavaScript avec Github Pages

1. Déposez votre projet dans un dépôt GitHub.
2. Dans les options du dépôt, activez GitHub Pages en sélectionnant la branche que vous souhaitez publier (généralement **main** ou **master**) et le dossier racine (/).
3. Cliquez sur "Save" pour enregistrer vos modifications.
4. Votre prototype sera accessible à l'adresse <https://<votre-nom-utilisateur>.github.io/<nom-du-repo>/>.

Attention: La page index.html doit être à la racine du dépôt pour que GitHub Pages fonctionne correctement.

Attention encore: Si vous utilisez des chemins relatifs dans votre code, assurez-vous qu'ils fonctionnent correctement une fois le projet déployé sur GitHub Pages.

Attention toujours: GitHub Pages ne gère pas PHP. Si votre prototype utilise PHP, vous devrez utiliser un autre service d'hébergement.

